



UACH
UNIVERSIDAD AUTÓNOMA DE
CHIHUAHUA



Universidad Autónoma De Chihuahua

Facultad De Ingeniería

Ingeniería En Ciencias De La Computación

Bases De Datos Avanzadas

2.53 Proyecto 1a. Evaluación

Omar Pereda Quezada (376900)

7CC2

Chihuahua, Chih 20/Septiembre/2026

CONSULTA DE USUARIO Y CREACION DE USUARIO NUEVO

```
=== BLOG · MENÚ PRINCIPAL ===
1. Usuarios
2. Artículos
3. Comentarios
4. Etiquetas
5. Categorías
0. Salir
Opción: 1

=== USUARIOS ===
1. Alta de usuario
2. Listar usuarios
3. Modificar usuario
4. Eliminar usuario
5. Contar artículos de un usuario
0. Volver
Opción: 2
  id  name          email                  n_articles  n_comments
  --  -
  1   Ana García    ana@example.com        2           1
  2   Luis Pérez    luis@example.com       1           2
  3   Omar Pereda   peredaomar97@gmail.com 0           0
(3 fila(s))

=== USUARIOS ===
1. Alta de usuario
2. Listar usuarios
3. Modificar usuario
4. Eliminar usuario
5. Contar artículos de un usuario
0. Volver
Opción: 1
Nombre: Janelly monreal
Email: monreal98@gmail.com
✓ Usuario creado con id 4

=== USUARIOS ===
1. Alta de usuario
2. Listar usuarios
3. Modificar usuario
4. Eliminar usuario
5. Contar artículos de un usuario
0. Volver
Opción: 2
  id  name          email                  n_articles  n_comments
  --  -
  1   Ana García    ana@example.com        2           1
  2   Luis Pérez    luis@example.com       1           2
  3   Omar Pereda   peredaomar97@gmail.com 0           0
  4   Janelly monreal monreal98@gmail.com    0           0
(4 fila(s))
```

Modificar usuario

```
=== USUARIOS ===
 1. Alta de usuario
 2. Listar usuarios
 3. Modificar usuario
 4. Eliminar usuario
 5. Contar artículos de un usuario
 0. Volver
Opción: 3
Id del usuario: 4
Nombre [Janely monreal]: Janely TORRES
Email [monrealt98@gmail.com]:
✓ Usuario actualizado
```

```
=== USUARIOS ===
 1. Alta de usuario
 2. Listar usuarios
 3. Modificar usuario
 4. Eliminar usuario
 5. Contar artículos de un usuario
 0. Volver
Opción: 2
```

id	name	email	n_articles	n_comments
1	Ana García	ana@example.com	2	1
2	Luis Pérez	luis@example.com	1	2
3	Omar Pereda	peredaomar97@gmail.com	0	0
4	Janely TORRES	monrealt98@gmail.com	0	0

(4 fila(s))

Eliminar usuario

```
=== USUARIOS ===
 1. Alta de usuario
 2. Listar usuarios
 3. Modificar usuario
 4. Eliminar usuario
 5. Contar artículos de un usuario
 0. Volver
Opción: 4
Id del usuario: 4
¿Borrar a Janely TORRES junto con sus artículos y comentarios? (s/N): s
✓ Usuario eliminado
```

```
=== USUARIOS ===
 1. Alta de usuario
 2. Listar usuarios
 3. Modificar usuario
 4. Eliminar usuario
 5. Contar artículos de un usuario
 0. Volver
Opción: 2
```

id	name	email	n_articles	n_comments
1	Ana García	ana@example.com	2	1
2	Luis Pérez	luis@example.com	1	2
3	Omar Pereda	peredaomar97@gmail.com	0	0

(3 fila(s))

Articulos por usuario

=== USUARIOS ===

1. Alta de usuario
2. Listar usuarios
3. Modificar usuario
4. Eliminar usuario
5. Contar artículos de un usuario
0. Volver

Opción: 5

Id del usuario: 1

El usuario 1 ha publicado 2 artículo(s).

Nuevo artículo y listar artículos

=== ARTÍCULOS ===

1. Alta de artículo
2. Listar artículos
3. Ver artículo completo
4. Modificar artículo
5. Eliminar artículo
6. Listar artículos de un usuario
7. Listar artículos por etiqueta
8. Listar artículos por categoría
9. Buscar artículos por texto
10. Añadir etiqueta a un artículo
11. Quitar etiqueta de un artículo
12. Añadir categoría a un artículo
13. Quitar categoría de un artículo
0. Volver

Opción: 1

Usuarios disponibles:

id	name	email	n_articles	n_comments
1	Ana García	ana@example.com	2	1
2	Luis Pérez	luis@example.com	1	2
3	Omar Pereda	peredaomar97@gmail.com	0	0

(3 fila(s))

Id del autor: 3

Título: Nuevo artículo

Contenido (termina con una línea vacía):

> ESTO ES UN NUEVO ARTICULO

> COONTINUACION...

>

✓ Artículo creado con id 4

=== ARTÍCULOS ===

1. Alta de artículo
2. Listar artículos
3. Ver artículo completo
4. Modificar artículo
5. Eliminar artículo
6. Listar artículos de un usuario
7. Listar artículos por etiqueta
8. Listar artículos por categoría
9. Buscar artículos por texto
10. Añadir etiqueta a un artículo
11. Quitar etiqueta de un artículo
12. Añadir categoría a un artículo
13. Quitar categoría de un artículo
0. Volver

Opción: 2

id	title	article_date	author	n_comments
4	Nuevo artículo	20/09/2026 22:46	Omar Pereda	0
3	Paquetes frente a procedimientos sueltos	20/09/2026 21:51	Luis Pérez	1
2	Conectar Python con Oracle	20/09/2026 21:51	Ana García	0

Artículo completo

=== ARTÍCULOS ===

1. Alta de artículo
2. Listar artículos
3. Ver artículo completo
4. Modificar artículo
5. Eliminar artículo
6. Listar artículos de un usuario
7. Listar artículos por etiqueta
8. Listar artículos por categoría
9. Buscar artículos por texto
10. Añadir etiqueta a un artículo
11. Quitar etiqueta de un artículo
12. Añadir categoría a un artículo
13. Quitar categoría de un artículo
0. Volver

Opción: 3

Id del artículo: 4

NUEVO ARTICULO

Por Omar Pereda – 20/09/2026 22:46

Etiquetas: –

Categorías: –

ESTO ES UN NUEVO ARTICULO

COONTINUACION...

Comentarios (0):
(sin resultados)

Listar artículos de un usuario

```
=== ARTÍCULOS ===
1. Alta de artículo
2. Listar artículos
3. Ver artículo completo
4. Modificar artículo
5. Eliminar artículo
6. Listar artículos de un usuario
7. Listar artículos por etiqueta
8. Listar artículos por categoría
9. Buscar artículos por texto
10. Añadir etiqueta a un artículo
11. Quitar etiqueta de un artículo
12. Añadir categoría a un artículo
13. Quitar categoría de un artículo
0. Volver
Opción: 6
Id del usuario: 3
  id  title                article_date    author      n_comments
  --  -
  4   Nuevo artículo      20/09/2026 22:46  Omar Pereda  0
(1 fila(s))

=== ARTÍCULOS ===
1. Alta de artículo
2. Listar artículos
3. Ver artículo completo
4. Modificar artículo
5. Eliminar artículo
6. Listar artículos de un usuario
7. Listar artículos por etiqueta
8. Listar artículos por categoría
9. Buscar artículos por texto
10. Añadir etiqueta a un artículo
11. Quitar etiqueta de un artículo
12. Añadir categoría a un artículo
13. Quitar categoría de un artículo
0. Volver
Opción: █
```

Etiqueta nueva a un artículo

```
=== ARTÍCULOS ===
1. Alta de artículo
2. Listar artículos
3. Ver artículo completo
4. Modificar artículo
5. Eliminar artículo
6. Listar artículos de un usuario
7. Listar artículos por etiqueta
8. Listar artículos por categoría
9. Buscar artículos por texto
10. Añadir etiqueta a un artículo
11. Quitar etiqueta de un artículo
12. Añadir categoría a un artículo
13. Quitar categoría de un artículo
0. Volver
Opción: 10
Id del artículo: 4
  id  name    url      n_articles
  --  -
  1   Oracle  oracle   2
  2   PL/SQL  pl-sql   2
  3   Python  python   1
(3 fila(s))
Id de la etiqueta a añadir: 2
✓ Etiqueta añadida
```

Agregar etiqueta a un articulo

```
=== ARTÍCULOS ===
1. Alta de artículo
2. Listar artículos
3. Ver artículo completo
4. Modificar artículo
5. Eliminar artículo
6. Listar artículos de un usuario
7. Listar artículos por etiqueta
8. Listar artículos por categoría
9. Buscar artículos por texto
10. Añadir etiqueta a un artículo
11. Quitar etiqueta de un artículo
12. Añadir categoría a un artículo
13. Quitar categoría de un artículo
0. Volver
Opción: 12
Id del artículo: 4
  id  name                url                n_articles
--  -
  1   Bases de datos     bases-de-datos     3
  2   Programación       programacion        1
(2 fila(s))
Id de la categoría a añadir: 1
✓ Categoría añadida
```

Consultar articulo completo con categoria y etiqueta nuevas

```
=== ARTÍCULOS ===
1. Alta de artículo
2. Listar artículos
3. Ver artículo completo
4. Modificar artículo
5. Eliminar artículo
6. Listar artículos de un usuario
7. Listar artículos por etiqueta
8. Listar artículos por categoría
9. Buscar artículos por texto
10. Añadir etiqueta a un artículo
11. Quitar etiqueta de un artículo
12. Añadir categoría a un artículo
13. Quitar categoría de un artículo
0. Volver
Opción: 3
Id del artículo: 4

NUEVO ARTICULO
Por Omar Pereda - 20/09/2026 22:46
Etiquetas:  PL/SQL
Categorías: Bases de datos

ESTO ES UN NUEVO ARTICULO
COONTINUACION...

Comentarios (0):
(sin resultados)
```

Buscar artículo

```
=== ARTÍCULOS ===
1. Alta de artículo
2. Listar artículos
3. Ver artículo completo
4. Modificar artículo
5. Eliminar artículo
6. Listar artículos de un usuario
7. Listar artículos por etiqueta
8. Listar artículos por categoría
9. Buscar artículos por texto
10. Añadir etiqueta a un artículo
11. Quitar etiqueta de un artículo
12. Añadir categoría a un artículo
13. Quitar categoría de un artículo
0. Volver
Opción: 9
Texto a buscar (título o contenido): Introducción a PL/SQL
  id  title                                     article_date      author          n_comments
--  -
  1   Introducción a PL/SQL   20/09/2026 21:51   Ana García     2
(1 fila(s))
```

Agregar un comentario y listar comentarios

```
=== COMENTARIOS ===
1. Alta de comentario
2. Listar comentarios de un artículo
3. Listar comentarios de un usuario
4. Modificar comentario
5. Eliminar comentario
6. Contar comentarios de un artículo
0. Volver
Opción: 1
Id del artículo: 1
Usuarios disponibles:
  id  name          email                      n_articles  n_comments
--  -
  1   Ana García    ana@example.com           2            1
  2   Luis Pérez    luis@example.com          1            2
  3   Omar Pereda   peredaomar97@gmail.com    1            0
(3 fila(s))
Id del usuario que comenta: 3
Comentario: me gusto bastante!
URL (opcional):
✓ Comentario creado con id 4

=== COMENTARIOS ===
1. Alta de comentario
2. Listar comentarios de un artículo
3. Listar comentarios de un usuario
4. Modificar comentario
5. Eliminar comentario
6. Contar comentarios de un artículo
0. Volver
Opción: 2
Id del artículo: 1
  id  name                                     url                                     author
--  -
  1   ¡Muy útil, gracias!
  2   Me gustaría ver un ejemplo con cursores.  https://example.com/cursores
  4   me gusto bastante!
(3 fila(s))
```

Listar comentarios de usuario y modificar comentario

```
(S-1110(3))

=== COMENTARIOS ===
1. Alta de comentario
2. Listar comentarios de un artículo
3. Listar comentarios de un usuario
4. Modificar comentario
5. Eliminar comentario
6. Contar comentarios de un artículo
0. Volver
Opción: 3
Id del usuario: 3
  id  name                url  article_id  article
  --  -
  4   me gusto bastante!   1    Introducción a PL/SQL
(1 fila(s))

=== COMENTARIOS ===
1. Alta de comentario
2. Listar comentarios de un artículo
3. Listar comentarios de un usuario
4. Modificar comentario
5. Eliminar comentario
6. Contar comentarios de un artículo
0. Volver
Opción: 4
Id del comentario: 4
Comentario [me gusto bastante!]: Me gusto normalii
URL:
✓ Comentario actualizado
```

Eliminar comentario

```
=== COMENTARIOS ===
1. Alta de comentario
2. Listar comentarios de un artículo
3. Listar comentarios de un usuario
4. Modificar comentario
5. Eliminar comentario
6. Contar comentarios de un artículo
0. Volver
Opción: 5
Id del comentario: 4
¿Borrar el comentario? (s/N): s
✓ Comentario eliminado

=== COMENTARIOS ===
1. Alta de comentario
2. Listar comentarios de un artículo
3. Listar comentarios de un usuario
4. Modificar comentario
5. Eliminar comentario
6. Contar comentarios de un artículo
0. Volver
Opción: 2
Id del artículo: 1
  id  name                url                author
  --  -
  1   ¡Muy útil, gracias!
  2   Me gustaría ver un ejemplo con cursores.  https://example.com/cursores  Luis Pérez
(2 fila(s))
```

Cantidad de comentarios de un artículo

```
=== COMENTARIOS ===
```

1. Alta de comentario
2. Listar comentarios de un artículo
3. Listar comentarios de un usuario
4. Modificar comentario
5. Eliminar comentario
6. Contar comentarios de un artículo
0. Volver

Opción: 6

Id del artículo: 1

El artículo 1 tiene 2 comentario(s).

Alta de etiqueta y listado de etiquetas

```
=== ETIQUETAS ===
```

1. Alta de etiqueta
2. Listar tags
3. Modificar etiqueta
4. Eliminar etiqueta
5. Contar artículos asociados
0. Volver

Opción: 1

Nombre: ETIQUETA

URL (vacío = generar a partir del nombre):

✓ Etiqueta creada con id 4

```
=== ETIQUETAS ===
```

1. Alta de etiqueta
2. Listar tags
3. Modificar etiqueta
4. Eliminar etiqueta
5. Contar artículos asociados
0. Volver

Opción: 2

id	name	url	n_articles
4	ETIQUETA	etiqueta	0
1	Oracle	oracle	2
2	PL/SQL	pl-sql	2
3	Python	python	1

(4 fila(s))

Modificar etiqueta y eliminar etiqueta

```
=== ETIQUETAS ===
```

1. Alta de etiqueta
2. Listar tags
3. Modificar etiqueta
4. Eliminar etiqueta
5. Contar artículos asociados
0. Volver

Opción: 3

Id de la etiqueta: 4

Nombre [ETIQUETA]: TAG

URL [etiqueta]:

✓ Etiqueta actualizada

```
=== ETIQUETAS ===
```

1. Alta de etiqueta
2. Listar tags
3. Modificar etiqueta
4. Eliminar etiqueta
5. Contar artículos asociados
0. Volver

Opción: 4

Id de la etiqueta: 4

¿Borrar 'TAG'? (se desvincula de sus artículos) (s/N): S

✓ Etiqueta eliminada

Contar artículos con cierta etiqueta

```
=== ETIQUETAS ===
```

1. Alta de etiqueta
2. Listar tags
3. Modificar etiqueta
4. Eliminar etiqueta
5. Contar artículos asociados
0. Volver

Opción: 5

Id de la etiqueta: 1

Tiene 2 artículo(s) asociado(s).

Alta de categoria y listado de categorías

```
=== CATEGORÍAS ===
1. Alta de categoría
2. Listar categories
3. Modificar categoría
4. Eliminar categoría
5. Contar artículos asociados
0. Volver
Opción: 1
Nombre: CATEGO
URL (vacío = generar a partir del nombre):
✓ Categoría creada con id 3

=== CATEGORÍAS ===
1. Alta de categoría
2. Listar categories
3. Modificar categoría
4. Eliminar categoría
5. Contar artículos asociados
0. Volver
Opción: 2
  id  name                url                n_articles
  --  -
  1   Bases de datos     bases-de-datos    3
  3   CATEGO             catego           0
  2   Programación       programacion      1
(3 fila(s))
```

Actualizar categoria y eliminar categoria

```
=== CATEGORÍAS ===
1. Alta de categoría
2. Listar categories
3. Modificar categoría
4. Eliminar categoría
5. Contar artículos asociados
0. Volver
Opción: 3
Id de la categoría: 3
Nombre [CATEGO]: categoriaaa
URL [catego]:
✓ Categoría actualizada

=== CATEGORÍAS ===
1. Alta de categoría
2. Listar categories
3. Modificar categoría
4. Eliminar categoría
5. Contar artículos asociados
0. Volver
Opción: 4
Id de la categoría: 3
¿Borrar 'categoriaaa'? (se desvincula de sus artículos) (s/N):
```

Contar artículos con cierta categoria

```
=== CATEGORÍAS ===
```

1. Alta de categoría
2. Listar categories
3. Modificar categoría
4. Eliminar categoría
5. Contar artículos asociados
0. Volver

```
Opción: 5
```

```
Id de la categoría: 1
```

```
Tiene 3 artículo(s) asociado(s).
```

CODIGO COMENTADO

```
import getpass
import os
import re
import sys
from datetime import datetime

import oracledb

conn = None # aquí se guarda la conexión a Oracle

#
=====
# Funciones para trabajar con PL/SQL
#
=====
class NotFound(Exception):
    """El registro pedido no existe."""

def connect():
    # se toman los datos de conexión y se abre la conexión con
    # Oracle
    user = os.getenv("BLOG_USER", "system")
    password = os.getenv("BLOG_PASSWORD", "oracle")
    dsn = os.getenv("BLOG_DSN", "localhost:1521/XEPDB1")
    return oracledb.connect(user=user, password=password,
    dsn=dsn)

def friendly(exc):
    # se obtiene el mensaje del error para mostrarlo de una forma
    # más clara
    err = exc.args[0]
    text = getattr(err, "message", str(err))
    code = getattr(err, "code", None)

    # los errores 20000 son errores creados desde PL/SQL
    if code and 20000 <= code <= 20999:
        return re.sub(r"^ORA-\d+:\s*", "", text.splitlines()[0])

    return text.strip()
```

```
def call_proc(name, *params):
    # llama a un procedimiento que no necesita regresar ningún
    valor
    with conn.cursor() as cur:
        cur.callproc(name, list(params))
```

```
def call_proc_out_id(name, *params):
    # se usa cuando el procedimiento crea un registro y devuelve
    su id
    with conn.cursor() as cur:
        new_id = cur.var(int)
        cur.callproc(name, [*params, new_id])
        return int(new_id.getvalue())
```

```
def call_func(name, return_type, *params):
    # llama a una función de PL/SQL y obtiene el resultado
    with conn.cursor() as cur:
        return cur.callfunc(name, return_type, list(params))
```

```
def _plain(row):
    # convierte algunos datos de Oracle para poder imprimirlos
    out = []
```

```
    for v in row:
        # los LOB necesitan leerse antes de mostrarlos
        if isinstance(v, oracledb.LOB):
            v = v.read()
        elif isinstance(v, datetime):
            # las fechas se muestran con un formato más sencillo
            v = v.strftime("%d/%m/%Y %H:%M")
        elif v is None:
            v = ""
```

```
    out.append(v)
```

```
    return out
```

```
def call_query(name, *params):
    # ejecuta una función que devuelve los resultados en un
    cursor
```

```
with conn.cursor() as cur:
    ref = cur.callfunc(name, oracledb.DB_TYPE_CURSOR,
list(params))
```

```
# se toman los nombres de las columnas
headers = [d[0] for d in ref.description]
```

```
# se guardan las filas que regresó Oracle
rows = [_plain(r) for r in ref.fetchall()]
```

```
ref.close()
```

```
return headers, rows
```

```
def fetch_one(name, id_, what="El registro"):
    # busca un solo registro usando su id
    headers, rows = call_query(name, id_)
```

```
# si no encuentra nada se manda el error correspondiente
if not rows:
    raise NotFound(f"{what} con id {id_} no existe.")
```

```
return dict(zip((h.lower() for h in headers), rows[0]))
```

```
#
```

```
=====
```

```
# Entrada / salida por consola
```

```
#
```

```
=====
```

```
def print_table(headers, rows, max_width=45):
    # si no hay datos no se imprime la tabla
    if not rows:
        print(" (sin resultados)")
        return
```

```
cells = []
```

```
for r in rows:
    line = []
```

```
    for c in r:
        s = str(c).replace("\n", " ")
```

```

        # evita que una columna demasiado larga desacomode la
tabla
        line.append(s if len(s) <= max_width else s[:
max_width - 1] + "...")

```

```

        cells.append(line)

```

```

        heads = [h.lower() for h in headers]

```

```

        # calcula el espacio que necesita cada columna
widths = [max(len(h), *(len(r[i]) for r in cells)) for i, h
in enumerate(heads)]

```

```

        print(" " + " ".join(h.ljust(w) for h, w in zip(heads,
widths)))
        print(" " + " ".join("-" * w for w in widths))

```

```

        # imprime cada fila de la tabla
for r in cells:
    print(" " + " ".join(c.ljust(w) for c, w in zip(r,
widths)))

```

```

        print(f" ({len(rows)} fila(s))")

```

```

def show(name, *params):
    # ejecuta una consulta y muestra sus resultados
    headers, rows = call_query(name, *params)
    print_table(headers, rows)

```

```

def ask(prompt, default=None, required=True):
    # pide un dato al usuario y permite usar un valor por defecto
    suffix = f" [{default}]" if default not in (None, "") else ""

```

```

    while True:
        value = input(f"{prompt}{suffix}: ").strip()

```

```

        if not value and default is not None:
            return default

```

```

        if value or not required:
            return value

```

```

        print(" Este campo es obligatorio.")

```

```
def ask_int(prompt):  
    # pide un número entero y vuelve a preguntar si se escribe  
    otra cosa
```

```
    while True:  
        value = input(f"{prompt}: ").strip()
```

```
        if value.isdigit():  
            return int(value)
```

```
    print(" Introduce un número entero.")
```

```
def ask_text(prompt, default=None):  
    # permite escribir contenido en varias líneas  
    hint = ("termina con una línea vacía; si no escribes nada se  
    conserva el actual"
```

```
            if default else "termina con una línea vacía")
```

```
    print(f"{prompt} ({hint}):")
```

```
    lines = []
```

```
    while True:  
        line = input(" > ")
```

```
        if not line:  
            break
```

```
        lines.append(line)
```

```
    text = "\n".join(lines).strip()
```

```
    if text:  
        return text
```

```
    if default:  
        return default
```

```
    print(" Este campo es obligatorio.")  
    return ask_text(prompt, default)
```

```
def confirm(prompt):  
    # solo se continúa cuando el usuario responde con s
```

```
return input(f"{prompt} (s/N): ").strip().lower() == "s"
```

```
def menu(title, options, back="Volver"):  
    # muestra las opciones del menú y ejecuta la que el usuario  
    elija
```

```
    while True:  
        print(f"\n=== {title} ===")
```

```
        for i, (label, _) in enumerate(options, 1):  
            print(f" {i}. {label}")
```

```
        print(f" 0. {back}")
```

```
        choice = input("Opción: ").strip()
```

```
        if choice == "0":  
            return
```

```
        if not (choice.isdigit() and 1 <= int(choice) <=  
len(options)):  
            print(" Opción no válida.")  
            continue
```

```
        try:  
            options[int(choice) - 1][1]()  
        except oracledb.DatabaseError as exc:  
            print(f" × {friendly(exc)}")  
        except NotFound as exc:  
            print(f" × {exc}")
```

```
#
```

```
=====
```

```
=====
```

```
# USUARIOS
```

```
#
```

```
=====
```

```
=====
```

```
def user_add():  
    # pide los datos y crea un usuario nuevo  
    name = ask("Nombre")  
    email = ask("Email")  
    uid = call_proc_out_id("pkg_users.add_user", name, email)  
    print(f" ✓ Usuario creado con id {uid}")
```

```
def user_list():  
    # muestra todos los usuarios  
    show("pkg_users.list_users")
```

```
def user_update():  
    # primero se busca el usuario para mostrar sus datos actuales  
    uid = ask_int("Id del usuario")  
    cur = fetch_one("pkg_users.get_user", uid, "El usuario")
```

```
    # se actualizan los datos que escriba el usuario  
    call_proc("pkg_users.update_user", uid,   
              ask("Nombre", cur["name"]), ask("Email",  
cur["email"])))
```

```
    print("  ✓ Usuario actualizado")
```

```
def user_delete():  
    # busca el usuario antes de eliminarlo  
    uid = ask_int("Id del usuario")  
    cur = fetch_one("pkg_users.get_user", uid, "El usuario")
```

```
    # pide confirmación porque también se eliminan sus datos  
    relacionados  
    if confirm(f"¿Borrar a {cur['name']} junto con sus artículos  
y comentarios?"):   
        call_proc("pkg_users.delete_user", uid)  
        print("  ✓ Usuario eliminado")
```

```
def user_count_articles():  
    # cuenta los artículos que tiene publicados un usuario  
    uid = ask_int("Id del usuario")  
    total = call_func("pkg_users.count_articles", int, uid)  
    print(f"  El usuario {uid} ha publicado {total}  
artículo(s).")
```

```
def users_menu():  
    menu("USUARIOS", [  
        ("Alta de usuario", user_add),  
        ("Listar usuarios", user_list),  
        ("Modificar usuario", user_update),  
        ("Eliminar usuario", user_delete),
```

```
        ("Contar artículos de un usuario", user_count_articles),  
    ])
```

```
#
```

```
=====
```

```
# ETIQUETAS y CATEGORÍAS
```

```
#
```

```
=====
```

```
def taxonomy_menu(title, pkg, one, many, label):
```

```
    def add():  
        # crea una etiqueta o categoría nueva  
        name = ask("Nombre")  
        url = ask("URL (vacío = generar a partir del nombre)",  
required=False) or None  
        new_id = call_proc_out_id(f"{pkg}.add_{one}", name, url)  
        print(f" ✓ {label} creada con id {new_id}")
```

```
    def lst():  
        # muestra todas las etiquetas o categorías  
        show(f"{pkg}.list_{many}")
```

```
    def update():  
        # busca primero el registro que se quiere modificar  
        tid = ask_int(f"Id de la {label.lower()}")  
        cur = fetch_one(f"{pkg}.get_{one}", tid, f"La  
{label.lower()}")
```

```
        call_proc(f"{pkg}.update_{one}", tid,  
ask("Nombre", cur["name"]), ask("URL",  
cur["url"]))
```

```
        print(f" ✓ {label} actualizada")
```

```
    def delete():  
        # busca el registro antes de eliminarlo  
        tid = ask_int(f"Id de la {label.lower()}")  
        cur = fetch_one(f"{pkg}.get_{one}", tid, f"La  
{label.lower()}")
```

```
        if confirm(f"¿Borrar '{cur['name']}'? (se desvincula de  
sus artículos)":  
            call_proc(f"{pkg}.delete_{one}", tid)
```

```
print(f"  ✓ {label} eliminada")
```

```
def count():  
    # cuenta los artículos relacionados con la etiqueta o  
    categoría  
    tid = ask_int(f"Id de la {label.lower()}")  
    total = call_func(f"{pkg}.count_articles", int, tid)  
    print(f"  Tiene {total} artículo(s) asociado(s).")
```

```
menu(title, [  
    (f"Alta de {label.lower()}", add),  
    (f"Listar {many}", lst),  
    (f"Modificar {label.lower()}", update),  
    (f"Eliminar {label.lower()}", delete),  
    ("Contar artículos asociados", count),  
])
```

```
def tags_menu():  
    # abre el menú de etiquetas  
    taxonomy_menu("ETIQUETAS", "pkg_tags", "tag", "tags",  
"Etiqueta")
```

```
def categories_menu():  
    # abre el menú de categorías  
    taxonomy_menu("CATEGORÍAS", "pkg_categories", "category",  
"categories", "Categoría")
```

```
#
```

```
# ARTÍCULOS
```

```
#
```

```
def article_add():  
    # muestra los usuarios para poder elegir al autor  
    print("Usuarios disponibles:")  
    show("pkg_users.list_users")
```

```
uid = ask_int("Id del autor")  
title = ask("Título")  
content = ask_text("Contenido")
```

```
# guarda el artículo y obtiene el id generado
aid = call_proc_out_id("pkg_articles.add_article", title,
content, uid)
print(f"  ✓ Artículo creado con id {aid}")
```

```
def article_list():
    # muestra todos los artículos
    show("pkg_articles.list_articles")
```

```
def article_view():
    # busca el artículo que se quiere consultar
    aid = ask_int("Id del artículo")
    a = fetch_one("pkg_articles.get_article", aid, "El artículo")
```

```
# obtiene las etiquetas y categorías del artículo
_, tags = call_query("pkg_articles.get_tags", aid)
_, cats = call_query("pkg_articles.get_categories", aid)
```

```
print(f"\n  {a['title'].upper()}")
print(f"  Por {a['author']} - {a['article_date']}")
print(f"  Etiquetas:  {'', '.join(t[1] for t in tags) or
'-'}")
print(f"  Categorías: {'', '.join(c[1] for c in cats) or '-'}
\n")
```

```
print(a["content"])
```

```
# al final se muestran los comentarios del artículo
print(f"\n  Comentarios ({a['n_comments']}):")
show("pkg_comments.list_by_article", aid)
```

```
def article_update():
    # busca el artículo para conservar sus datos si no se cambian
    aid = ask_int("Id del artículo")
    cur = fetch_one("pkg_articles.get_article", aid, "El
artículo")
```

```
title = ask("Título", cur["title"])
content = ask_text("Contenido", cur["content"])
```

```
call_proc("pkg_articles.update_article", aid, title, content)
print(f"  ✓ Artículo actualizado")
```

```
def article_delete():
    # busca el artículo antes de eliminarlo
    aid = ask_int("Id del artículo")
    cur = fetch_one("pkg_articles.get_article", aid, "El
artículo")
```

```
    if confirm(f"¿Borrar '{cur['title']}' y sus
{cur['n_comments']} comentario(s)?"):
        call_proc("pkg_articles.delete_article", aid)
        print(" ✓ Artículo eliminado")
```

```
def article_by_user():
    # muestra los artículos publicados por un usuario
    show("pkg_articles.list_by_user", ask_int("Id del usuario"))
```

```
def article_by_tag():
    # primero se muestran las etiquetas disponibles
    show("pkg_tags.list_tags")
    show("pkg_articles.list_by_tag", ask_int("Id de la
etiqueta"))
```

```
def article_by_category():
    # primero se muestran las categorías disponibles
    show("pkg_categories.list_categories")
    show("pkg_articles.list_by_category", ask_int("Id de la
categoría"))
```

```
def article_search():
    # busca artículos que tengan el texto indicado
    show("pkg_articles.search_articles", ask("Texto a buscar
(título o contenido)"))
```

```
def article_add_tag():
    # agrega una etiqueta al artículo seleccionado
    aid = ask_int("Id del artículo")
    show("pkg_tags.list_tags")
    call_proc("pkg_articles.add_tag", aid, ask_int("Id de la
etiqueta a añadir"))
    print(" ✓ Etiqueta añadida")
```

```
def article_remove_tag():
    # muestra las etiquetas actuales antes de quitar una
    aid = ask_int("Id del artículo")
    show("pkg_articles.get_tags", aid)
    call_proc("pkg_articles.remove_tag", aid, ask_int("Id de la
etiqueta a quitar"))
    print(" ✓ Etiqueta quitada")
```

```
def article_add_category():
    # agrega una categoría al artículo
    aid = ask_int("Id del artículo")
    show("pkg_categories.list_categories")
    call_proc("pkg_articles.add_category", aid, ask_int("Id de la
categoría a añadir"))
    print(" ✓ Categoría añadida")
```

```
def article_remove_category():
    # muestra las categorías actuales antes de quitar una
    aid = ask_int("Id del artículo")
    show("pkg_categories.get_categories", aid)
    call_proc("pkg_articles.remove_category", aid, ask_int("Id de
la categoría a quitar"))
    print(" ✓ Categoría quitada")
```

```
def articles_menu():
    menu("ARTÍCULOS", [
        ("Alta de artículo", article_add),
        ("Listar artículos", article_list),
        ("Ver artículo completo", article_view),
        ("Modificar artículo", article_update),
        ("Eliminar artículo", article_delete),
        ("Listar artículos de un usuario", article_by_user),
        ("Listar artículos por etiqueta", article_by_tag),
        ("Listar artículos por categoría", article_by_category),
        ("Buscar artículos por texto", article_search),
        ("Añadir etiqueta a un artículo", article_add_tag),
        ("Quitar etiqueta de un artículo", article_remove_tag),
        ("Añadir categoría a un artículo", article_add_category),
        ("Quitar categoría de un artículo",
article_remove_category),
    ])
    ])
```

```

#
=====
# COMENTARIOS
#
=====
def comment_add():
    # primero se pide el artículo donde se va a poner el
    comentario
    aid = ask_int("Id del artículo")

    print("Usuarios disponibles:")
    show("pkg_users.list_users")

    # se elige el usuario que hará el comentario
    uid = ask_int("Id del usuario que comenta")
    name = ask("Comentario")
    url = ask("URL (opcional)", required=False) or None

    cid = call_proc_out_id("pkg_comments.add_comment", name, url,
uid, aid)
    print(f"  ✓ Comentario creado con id {cid}")

def comment_list_by_article():
    # muestra todos los comentarios de un artículo
    show("pkg_comments.list_by_article", ask_int("Id del
artículo"))

def comment_list_by_user():
    # muestra los comentarios que ha hecho un usuario
    show("pkg_comments.list_by_user", ask_int("Id del usuario"))

def comment_update():
    # busca el comentario para mostrar sus datos actuales
    cid = ask_int("Id del comentario")
    cur = fetch_one("pkg_comments.get_comment", cid, "El
comentario")

    name = ask("Comentario", cur["name"])
    url = ask("URL", cur["url"], required=False) or None

```

```
call_proc("pkg_comments.update_comment", cid, name, url)
print("  ✓ Comentario actualizado")
```

```
def comment_delete():
    # verifica que el comentario exista antes de borrarlo
    cid = ask_int("Id del comentario")
    fetch_one("pkg_comments.get_comment", cid, "El comentario")
```

```
    if confirm("¿Borrar el comentario?"):
        call_proc("pkg_comments.delete_comment", cid)
        print("  ✓ Comentario eliminado")
```

```
def comment_count():
    # cuenta los comentarios que tiene un artículo
    aid = ask_int("Id del artículo")
    total = call_func("pkg_articles.count_comments", int, aid)
    print(f"  El artículo {aid} tiene {total} comentario(s).")
```

```
def comments_menu():
    menu("COMENTARIOS", [
        ("Alta de comentario", comment_add),
        ("Listar comentarios de un artículo",
comment_list_by_article),
        ("Listar comentarios de un usuario",
comment_list_by_user),
        ("Modificar comentario", comment_update),
        ("Eliminar comentario", comment_delete),
        ("Contar comentarios de un artículo", comment_count),
    ])
```

```
#
```

```
=====
# Programa principal
```

```
#
```

```
=====
def main():
    global conn
```

```
    # intenta conectarse a Oracle antes de mostrar el menú
    try:
```

```
    conn = connect()
except oracledb.Error as exc:
    print(f"No se pudo conectar a Oracle: {friendly(exc)}")
    sys.exit(1)
```

```
print(f"Conectado a Oracle {conn.version}")
```

```
try:
    # muestra el menú principal del programa
    menu("BLOG · MENÚ PRINCIPAL", [
        ("Usuarios", users_menu),
        ("Artículos", articles_menu),
        ("Comentarios", comments_menu),
        ("Etiquetas", tags_menu),
        ("Categorías", categories_menu),
    ], back="Salir")
```

```
except (KeyboardInterrupt, EOFError):
    print()
```

```
finally:
    # cierra la conexión cuando termina el programa
    conn.close()
    print("Hasta pronto.")
```

```
if __name__ == "__main__":
    main()
```